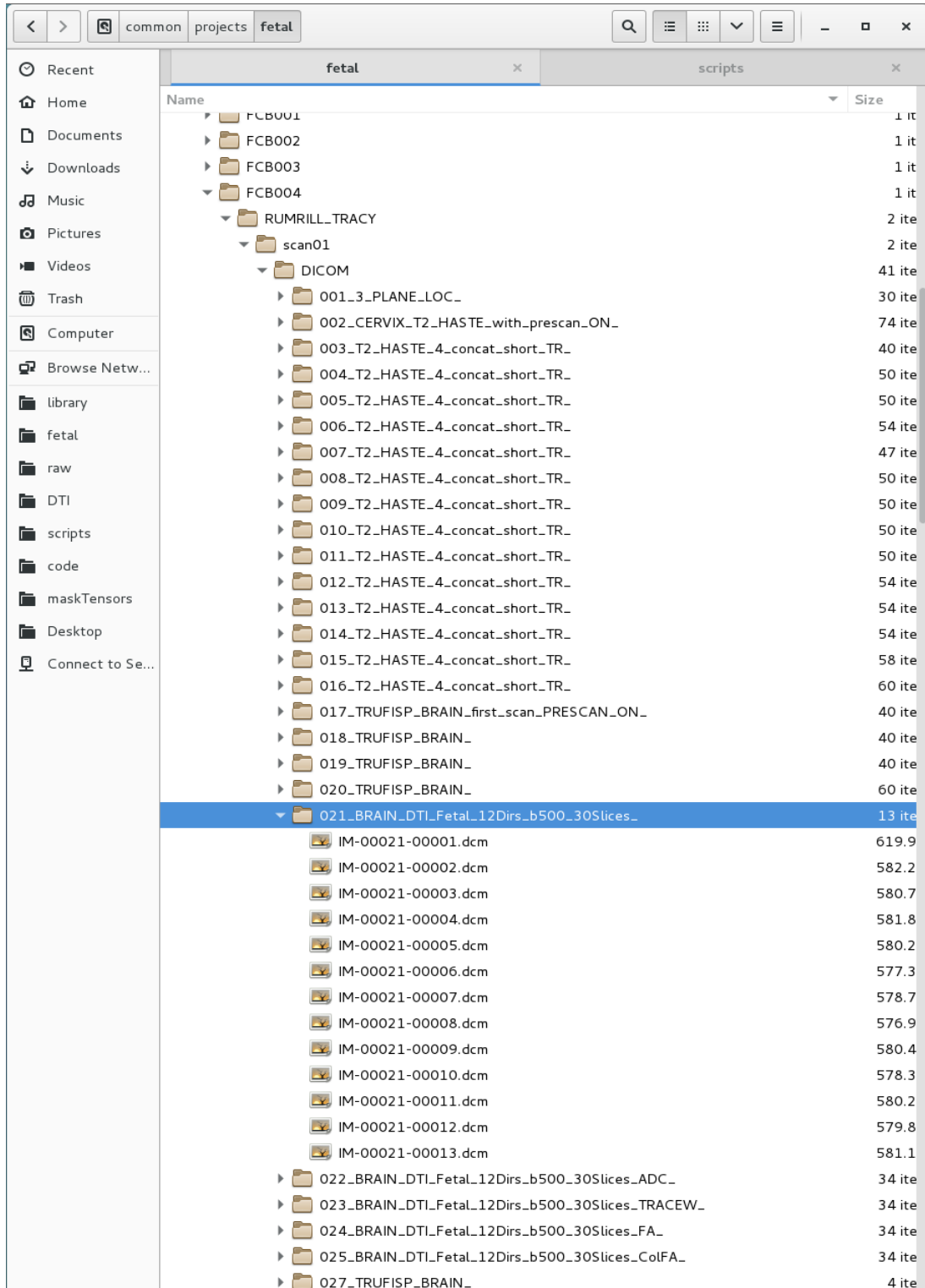


CRL FETAL DTI PROCESSING PIPELINE

Pipeline requires: Bash, Python3, Additional Python libraries: nibabel

STEP 1: IMPORT DATA FROM RAW REPOSITORY

- 1) Go to data location: /common/projects/fetal/raw OR /common/projects/fetal/FCB/raw or wherever else it might be stored.
- 2) Identify valid Brain DTI B0 Scans – These should have 13 dicom images inside their folder



Immediately after DWI data, you will see similarly named folder with suffix ADC, TRACEW, FA etc so it is actually easy to spot the DWI volumes.

014_T2_HASTE_4_concat_short_TR_	54 ite
015_T2_HASTE_4_concat_short_TR_	58 ite
016_T2_HASTE_4_concat_short_TR_	60 ite
017_TRUFISP_BRAIN_first_scan_PRESCAN_ON_	40 ite
018_TRUFISP_BRAIN_	40 ite
019_TRUFISP_BRAIN_	40 ite
020_TRUFISP_BRAIN_	60 ite
021_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_	13 ite
022_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_ADC_	34 ite
023_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_TRACEW_	34 ite
024_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_FA_	34 ite
025_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_ColFA_	34 ite
027_TRUFISP_BRAIN_	4 ite
028_TRUFISP_BRAIN_	10 ite
029_TRUFISP_BRAIN_	10 ite
030_TRUFISP_BRAIN_	5 ite
031_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_	13 ite
032_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_ADC_	34 ite
033_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_TRACEW_	34 ite
034_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_FA_	34 ite
035_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_ColFA_	34 ite
037_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_	13 ite
038_BRAIN_DTI_Fetal_12Dirs_b500_30Slices_ADC_	34 ite

2) Once the DWI DICOMs for a case have been located, we prepare to import them for our project:

```
cd /common/projects/Shadab/fetal/Brain
```

```
sh /home/ch191070/scripts/fetalDTI/dtiTemplate.sh CASEIDscanNum
```

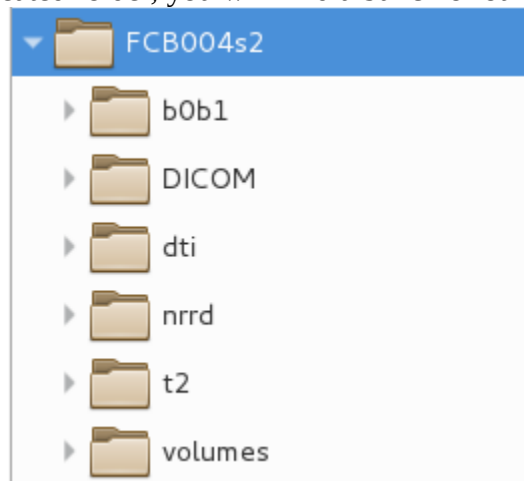
CASEIDscanNum is formatted as this: 1112s1 or 1112s2 for case number 1112 scan 1 or 2. Notice the lack of underscore or spaces between caseID and scan number. For FCB cases, we have used the following format: FCB087s2, FCB105s1 etc.

CASEIDscanNum **MUST** be in standard format. **Ensure this!**

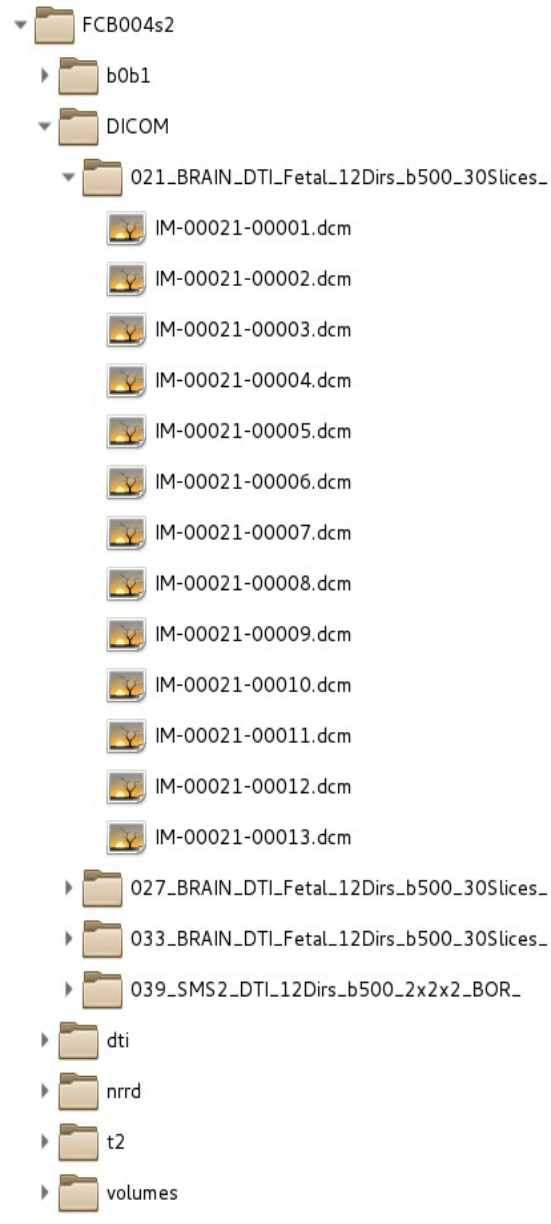
Once you run the script above (dtiTemplate.sh), a new folder will be created. In this example, we would create a new folder with the path for CASEIDscanNum = FCB004s2:

```
/common/projects/Shadab/fetal/Brain/FCB004s2
```

If you look inside the newly created folder, you will find a bunch of other empty folders:



We are now ready to import the DWI DICOMs. Copy the DICOM **folders** as is from the raw repository **into** the FCB004s2/DICOM folder:



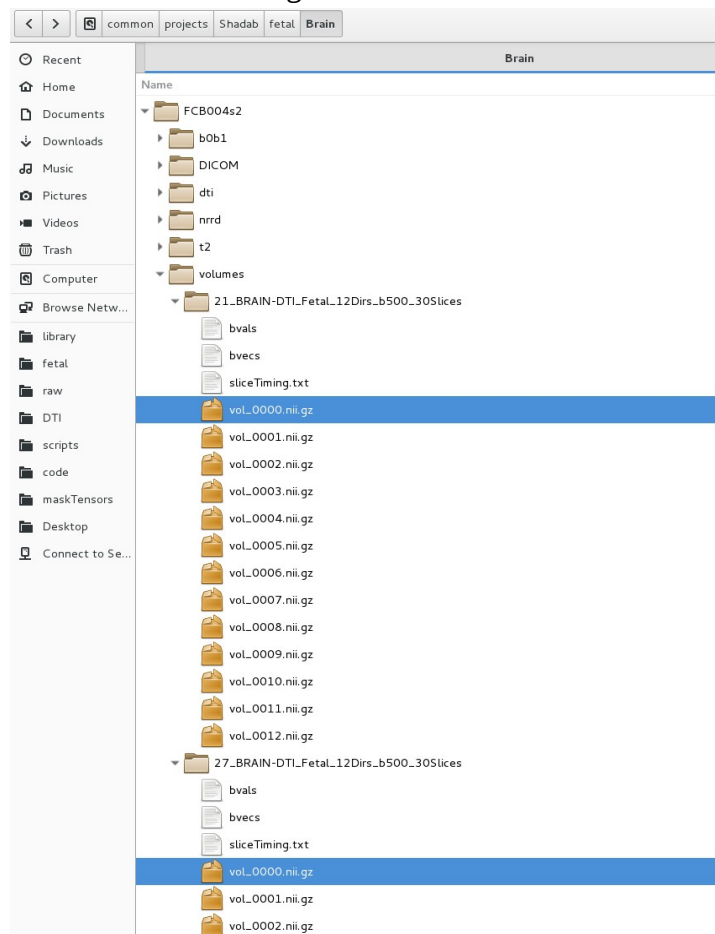
STEP 2: CONVERT DICOM AND CREATE REFERENCE FOR DTI COMPUTATION

1) We have a python script 'prepDWI.py' that takes care of this automatically. Simply call the script with the location of CASEIDscanNum (case data folder) and it will do the rest. Here is how it works:

```
export SK_DTI_SCRIPT_DIR=/home/ch191070/scripts/fetalDTI
python3 $SK_DTI_SCRIPT_DIR/prepDWI.py /common/projects/Shadab/fetal/Brain/FCB004s2
```

Here with the first line we tell the shell where the DTI scripts are located. Second line calls a Python script that first creates .nrrd files corresponding to the DICOMs. Then, it converts the nrrd files to NIFTI (.nii.gz). NRRD format files are located in FCB004s2/nrrd and NIFTI format files are located in FCB004s2/volumes. Python script also reads the b-values, b-vectors and slice timing off of the DICOMs and puts them in the corresponding folders in FCB004s2/volumes along with NIFTI images.

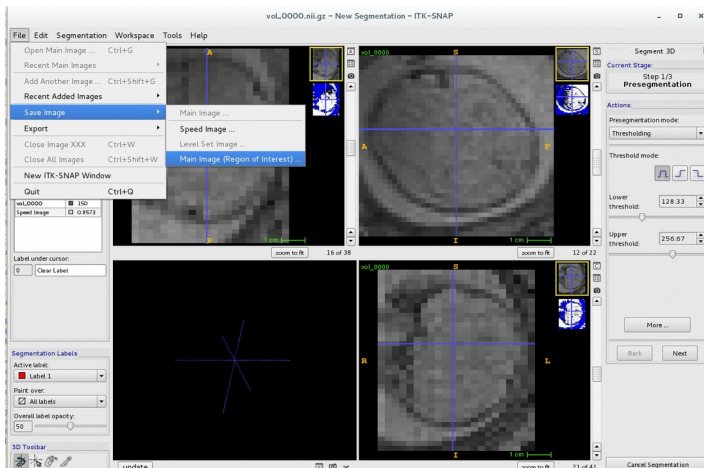
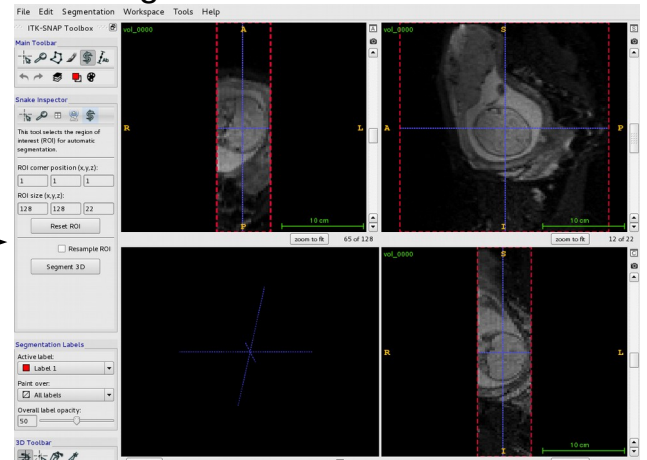
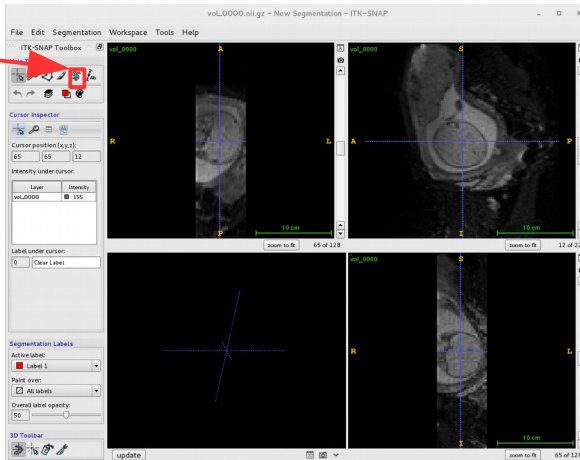
2) We create composite B0 (average of all b=0 images) image as part of the DTI. This step requires that we first create a reference for composite B0. To do this, we will look at all available b=0 images in ITK-SNAP and crop the one that looks good. First, choose b=0 volumes and open them in ITK SNAP. B=0 images are named vol_0000.nii.gz:



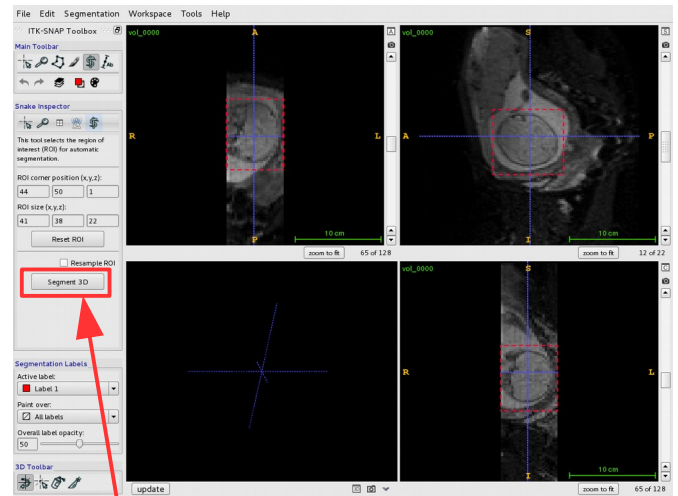
3) Once we've selected the b=0 image that appears to be best (free of motion artifact or significant intensity inhomogeneity), we proceed to crop it. This is also a good time to check for extreme intensity inhomogeneities or weird distortions. We do not correct for distortion at present so if anything looks too weird, drop that image set. If all image sets are bad, we do not process that case.

Adjust the crop boundaries using mouse cursor

Click Here



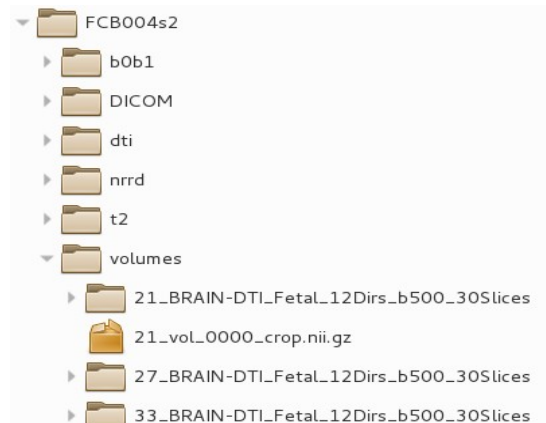
Save the image to disk as 'cropped.nii.gz' or a similar name



Click on 'Segment 3D'

4) Save the cropped image inside CASEIDscanNum/volumes as shown in the image to the right →

Note the file name format, XX_vol_0000_crop.nii.gz where XX represents the folder in CASEIDscanNum/volumes from where this cropped b=0 image came.



STEP 3: COMPUTE B0, B1

1) This step uses a Python script located at:

/home/ch191070/scripts/fetalDTI/createB0B1.py

The above script requires that an additional python3 compatible library (nibabel) be installed in the system. The above script in turn uses two other python scripts:

/home/ch191070/scripts/fetalDTI/createCompositeDiffusionImage.py

/home/ch191070/scripts/fetalDTI/b0b1ReconLib.py

and PSF profiles available at:

/home/ch191070/scripts/fetalDTI/PSFprofiles/

We call this script as below:

```
python3.5 /home/ch191070/scripts/fetalDTI/createB0B1.py  
/common/projects/Shadab/fetal/Brain/1147s1/volumes2/21_vol_0000_crop.nii.gz
```

The above line is one continuous command that we enter in the terminal. It appears in two lines because it can't be fit into one. There are only single spaces between each block. The black block ('python3.5') is the python executable, the **red block** is the location of the script we want to execute to compute B0 and B1 images. The **blue block is the argument** we provide to the **createB0B1.py** script.

What does this script do?

Many things. The script internally handles a lot of stuff, and it may be worth understanding what is going on. It should be possible to go through Python script to understand what is going on.

Output:

This script will generate a bunch of files and put them into the CASEIDScanNum/b0b1 folder. Among these files should be:

```
dwi_b0_CASEIDScanNum.nii.gz  
dwi_b0_CASEIDScanNum_crop.nii.gz  
dwi_b0_CASEIDScanNum_tensor.nii  
dwi_b1_CASEIDScanNum.nii.gz  
dwi_b1_CASEIDScanNum_crop.nii.gz
```

Before I explain what these files are, notice the naming convention. First thing is the physical space where these images are. We use 'dwi' to denote that these are in the space where all DWI images are. Second things tells you what type of image it is 'b0' or 'b1'. Third thing is the case and scan identifier. Finally, some of them have a fourth thing which gives you additional info. We stick to this convention for the most part. For example, you might see **t2_b0_CASEIDScanNum.nii.gz** or **atlas_b0_CASEIDScanNum.nii.gz** which tells you that the **b0** image for **CASEIDSanNum** case is in **t2** or **atlas** space. See below for an explanation of what these files are.

We create two versions of B0 (let's call them **B0_1** and **B0_2**) and one version of **B1**. **B0_1** and **B1** are used for registration purposes - these are the output of the superresolution process from Bernhard Kainz et al.'s code that we have modified extensively. **B0_2** is the version that is used for computing tensor - this is a result of forward projection of all b=0 images that is computed with a PSF formulation. Because the iterative updates apply a non-linear filter and modify the intensity content, we do not use **B0_1** for tensor computation even though it appears visually more appealing.

Also, the superresolution pipeline results in edge slices having some artifacts, that's why we have a cropped version of B0_1 and B1 for avoiding errors in cost computation during registration.

With this explanation, [B0_1](#) = dwi_b0_CASEIDScanNum_crop.nii.gz

[B1](#) = dwi_b1_CASEIDScanNum_crop.nii.gz

and [B0_2](#) = dwi_b0_CASEIDScanNum_tensor.nii

2) Sometimes, the createB0B1.py script fails. Actually, it's the superresolution pipeline that fails since the python script is just doing convenience tasks. This may happen because of various reasons. If the script fails, there will be nothing in [CASEIDScanNum/b0b1](#) folder. In this case, you will see some temporary folders inside [CASEIDScanNum/volumes](#). Go inside [CASEIDScanNum/volumes/tmpB0](#) folder and take a look at the images. We use

[CASEIDScanNum/volumes/tmpB0/GaussianReconstruction_GPU3.nii](#) for [B0_2](#) and

[CASEIDScanNum/volumes/tmpB0/image1_GPU.nii.gz](#) for [B0_1](#). Are these files available? How do they look? If they look fine, copy them into [CASEIDScanNum/b0b1](#) folder with the naming conventions defined above. Similarly, take a look inside [CASEIDScanNum/tmpB1](#). We use

[CASEIDScanNum/volumes/tmpB1/image1_GPU.nii.gz](#) for [B1](#), if this file is available but does not look good, we can use [CASEIDScanNum/volumes/tmpB1/GaussianReconstruction_GPU3.nii](#) as a replacement.

In a rare occasion when both B0 and B1 images couldn't be successfully computed, we drop the case from any further processing.

Quality Check: Open B0_1 and B1 in ITK-SNAP. Visually inspect to ensure that images are well aligned and free of any obvious problems. We use B0_1 as a reference for computing B1 so these two images should already be registered to each other. If this is not true, we drop the case from any further processing. This is rare, in my processing of >100 cases it happened only once.

STEP 4: PREPARE T2 DATA SET

1) Locate the T2 data of a case, this is generally in either of these two locations:

/common/external/crl-rdynaspro8/ali/CRL/[CASEID/ScanNum/nii](#)

OR /common/external/crl-rdynaspro8/ali/FCB/[CASEID/ScanNum/nii](#)

Navigate inside the folder and run the script below, the arguments to the scripts have been described:

[sh /home/ch191070/scripts/fetalDTI/createAtlasT2andMaskFile.sh](#)

[mask_cr3Drecon_CASEIDScanNum.nii](#) [register_CASEIDScanNum .nii.gz](#)

[tfm_nxbmcr3Drecon_CASEIDScanNum_FLIRTto_SOME_REF_FILE.txt](#) [CASEIDScanNum](#)

[fetus_XX.nii](#)

for example:

[sh /home/ch191070/scripts/fetalDTI/createAtlasT2andMaskFile.sh](#) [mask_cr3Drecon_f1196_s1.nii](#)

[register_f1196_s1.nii.gz](#) [tfm_nxbmcr3Drecon_f1196_s1_FLIRTto_FCB033_s2.txt](#) [1196s1](#)

[fetus_018.nii](#)

Hakim provides the files in [red](#).

HOW WAS fetus_018.nii SELECTED? To select a fetus_XX.nii file for the last argument, we locate the r3DreconOfetus_best_XX.nii file in the folder, see the green box in the image in the next page. We chose the corresponding fetus_XX.nii file. NOTE that we DO NOT pass the r3DreconOfetus file but the fetus_xx.nii file as an argument. Once the script is done, copy the resulting five files to [CASEIDScanNum/t2](#). The five files are:

[atlas_mask_CASEIDScanNum.nii.gz](#)

[atlas_t2_CASEIDScanNum.nii.gz](#)

[t2_mask_CASEIDScanNum.nii.gz](#)

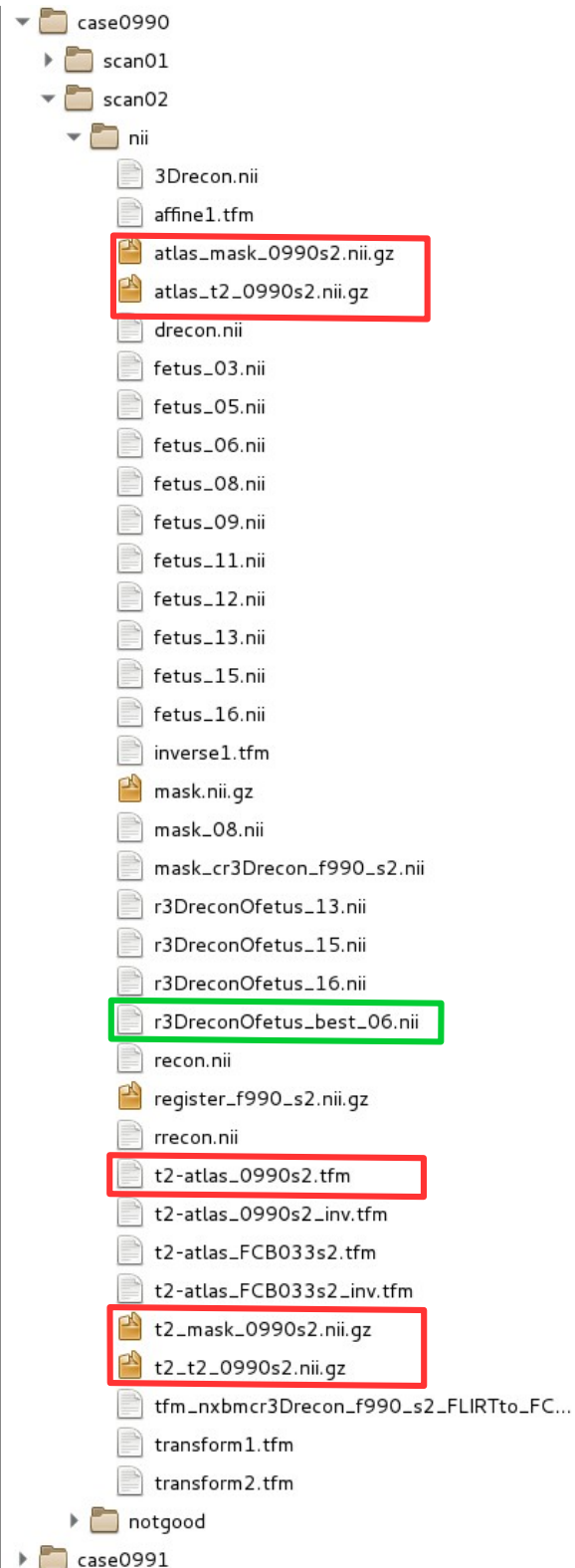
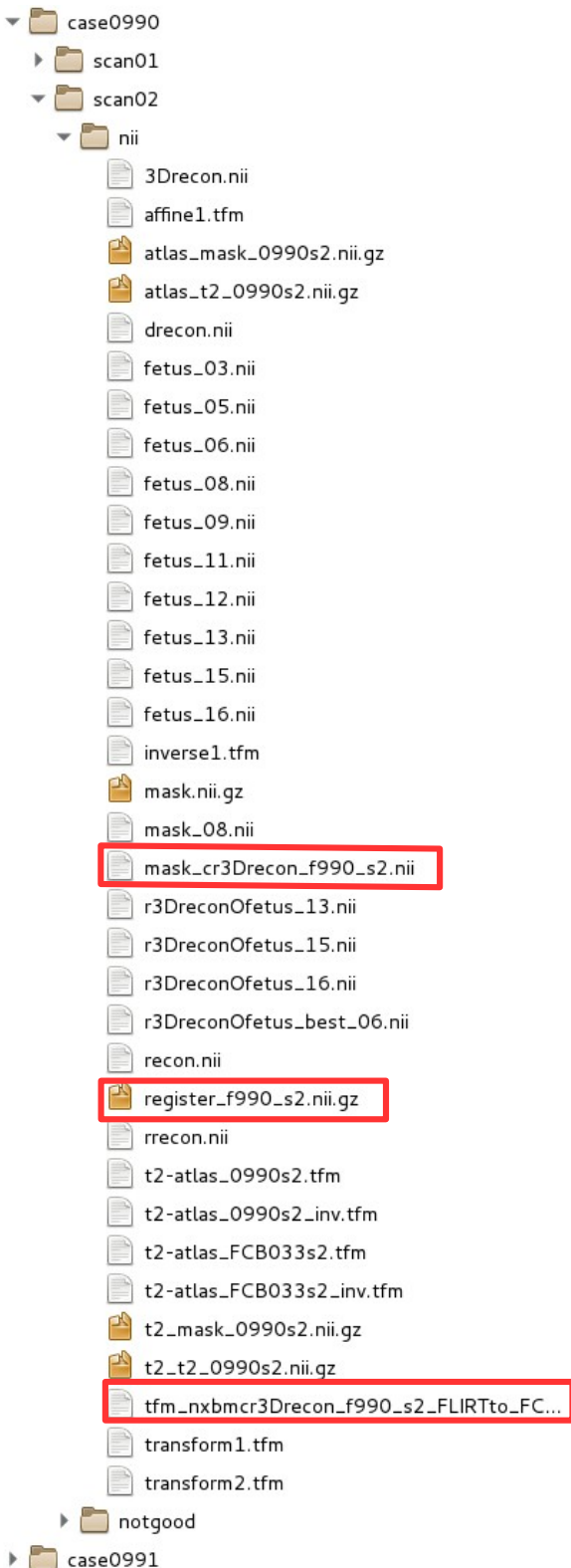
[t2_t2_CASEIDScanNum.nii.gz](#)

[t2-atlas_CASEIDScanNum.tfm](#)

We're now ready for next step.

On the LEFT : Hakim will give these three files for any case.

On the RIGHT : Copy the five files inside the red boxes. The file in the **green box** is marked by hakim and is used to decide the fetus_XX.nii argument in running the script in the previous page.



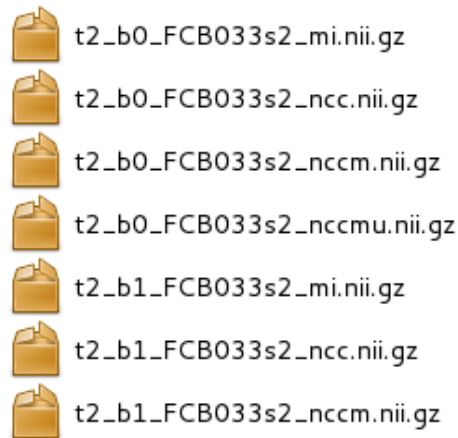
STEP 5: REGISTER B0/B1 WITH T2

In this step we register B0/B1 files created in step 3 with t2_t2_CASEIDScanNum.nii.gz file created in previous step.

```
python3.6 /home/ch191070/scripts/fetalDTI/registerB0B1toT2.py  
/common/projects/Shadab/fetal/Brain/CASEIDScanNum/b0b1/dwi b0 CASEIDScanNum  
_nii.gz
```

First argument is the b0 file that we created. It must be located inside the b0b1 folder and must have .nii.gz file extension. Use gzip to compress it to .gz extension if it is in .nii format.

Once the script finishes executing, there will be several files in CASEIDScanNum/b0b1 folder:



In this screenshot, we are looking at the files produced by the above script for FCB033s2. The filenames follow the usual pattern: t2 specifies that the image is in t2 space (registered to t2_t2_CASEIDScanNum.nii.gz). , b0 or b1 tells us whether it is B0 or B1 image that we had computed previously. The last term tells us the registration metric that was used and whether we used a mask, and if a mask was used, whether it dilated or undilated. Here is how to understand it:

mi → Mutual Information (No mask)

ncc → Normalized Cross Correlation (No mask)

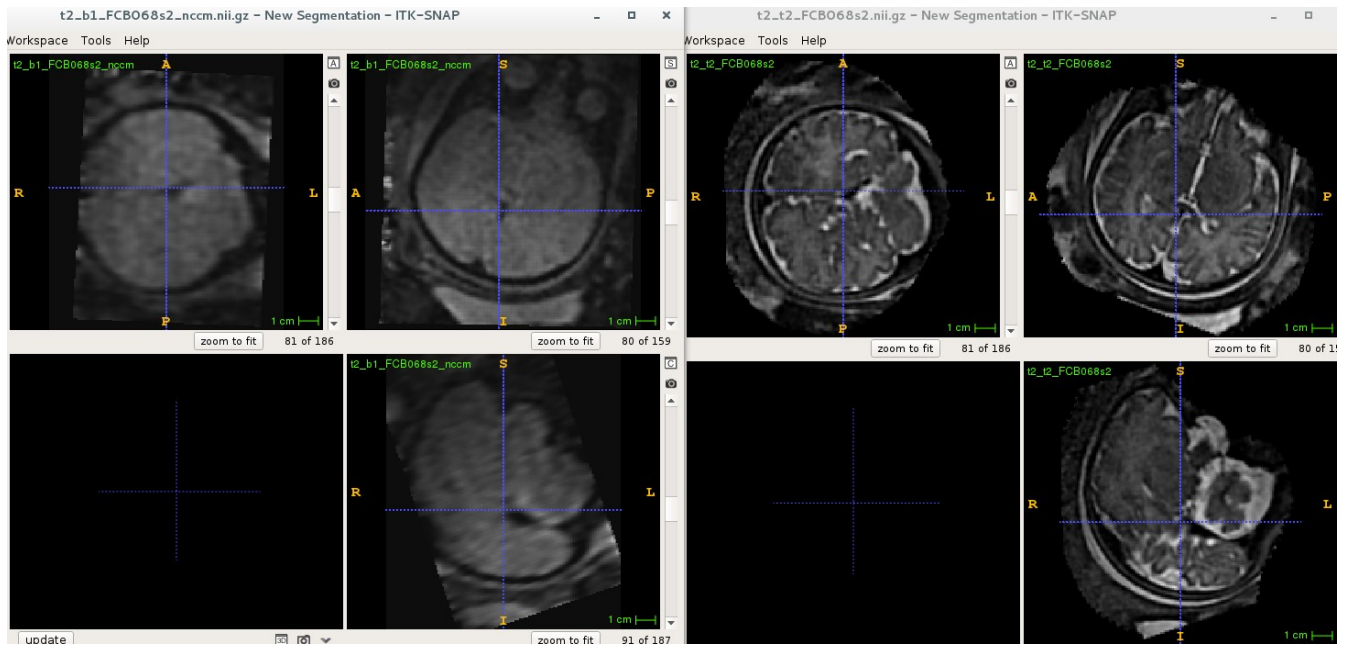
nccm → Normalized Cross Correlation with dilated mask.

nccmu → Normalized Cross Correlation with undilated mask.

Similarly we might have mim, mimu etc.

WE NEED TO: carefully check which of these registered well with t2. We do this by sequentially loading the t2_b0_.. or t2_b1_.. files and the corresponding t2_t2_CASEIDScanNum.nii.gz file in separate ITK-SNAP windows. In the next image, we are looking at b1 in t2 space for FCB068s2 case. We found that B1 image registered with t2 image using nccm metric led to the best registration between t2 and B0/B1 space (recall that B0/B1 are colocated, i.e. they're mutually aligned by default).

Once we have found the best image (either B0 or B1) and the best metric (mi, ncc, nccm, nccmu...) that led to the best registration between B0/B1 and t2 space, we make note of this in an excel sheet where we are tracking the progress of each case. We MUST record this information accurately and exactly in the following manner: As in the next image, if B1 image with nccm metric led to the best registration, record **b1nccm** in the excel sheet. If B0 image with mi led to the best registration, record **b0mi** in the excel sheet. The tensor compute program uses this information to pull correct transformation files and produce the computed tensor image.



A	B	C	D	E	F	G	H	
CASE ID	B0	B1	B0 Tensor	Registration	SVR	Tensor	Tractography	
0633s1	Y	Y	Y	b1mi	Y	Y	Y	
0650s1								
0779s1	Y	Y	Y	b0mi	Y	Y	Y	
0792s1	Y	Y	Y	b1mi	Y	Y	Y	
0826s1	Y	Y	Y	b0mi	Y	Y	Y	
0847s1	Y	Y	Y	b1mi	Y	Y	Y	
0853s2	Y	Y	Y	b1mi	Y	Y	Y	
0867s1	Y	Y	Y	b1nccm				
0906s2	Y	Y	Y	b1mi	Y	Y	Y	
0954s1	Y	Y	Y	b0ncc	Y	Y	Y	
0968s1	Y	Y	Y	b0mi	Y	Y	Y	
0968s2	Y	Y	Y	b1mi	Y	Y	Y	
0978s1	Y	Y	Y	b0mi	Y	Y	Y	
0980s2	Y	Y	Y	b0mi	Y	Y	Y	

STEP 6: SLICE TO VOLUME REGISTRATION AND TENSOR COMPUTE

Here we first register each slice from b=0 and b!=0 volumes to B0/B1 volumes computed in step 3. To do this, run the following script:

```
python3.6 /home/ch191070/scripts/fetalDTI/doSVRandTensorCompute.py
/common/projects/Shadab/fetal/Brain/CASEIDScanNum/b0b1/dwi_b0 CASEIDScanNum
.nii.gz auster b0mi
```

2nd last t argument specifies which binary to use: Shadab compiled these codes for each of the server machines. To run these programs on any of these machines, simply pass in the name of the machine as an argument from one of the following: auster, zephyr, boreas, dingo, bayes, eurus, io, lasker

Last argument is the suffix of the file name that appears most well aligned with the atlas_t2 file as identified in step 5.

STEP 7: REFINE TENSOR MASK

Tensor image will be generated and put in [CASEIDScanNum/dti](#) directory. In this directory, you will also see an FA (Fractional Anisotropy) and colored FA (RGB) images corresponding to the tensor image. Open the FA image in ITK-SNAP and inspect, does the image look like it has content, does it show the usual shape of the brain? Note that the area outside the brain region would look very noisy and the FA image at this stage will look really awful. Don't worry about this. If the resulting image is empty, inspect the color FA and tensor maps. If they also look empty, the tensor computation has failed for some reason.

If the tensor computation fails, reduce the dilation factor of mask from 4 to 2 or 1 in the script:
[/home/ch191070/scripts/fetalDTI/doSVRandTensorCompute.py](#)

We do this by passing an extra argument [-dilateMask](#) to the script in the manner shown below:

```
python3.6 /home/ch191070/scripts/fetalDTI/doSVRandTensorCompute.py  
/common/projects/Shadab/fetal/Brain/CASEIDScanNum/b0b1/dwi b0 CASEIDScanNum  
.nii.gz auster b0mi -dilateMask=2
```

In this example, dilation factor is set to 2, default is 4. If the tensor computation fails still, we drop this case from further processing.

Next step is to refine tensor mask. To do this, we open the B0 and B1 images in atlas space from [CASEIDScanNum/b0b1](#) and use these to create a brain mask. We start with the atlas_mask_CASEIDScanNum_1pt2.nii.gz image and modify it until it fits the atlas_B0... and atlas_B1... image. Hakim has done this before and he can help with doing this. Once the mask is ready, place it in [CASEIDScanNum/t2](#) directory with this exact naming scheme:
[atlas_mask_CASEIDScanNum_1pt2_refine.nii.gz](#).

The next step here is to multiply this mask with the tensor image to produce the final tensor image to be used for further analysis. We do it using the script below:

STEP 8: OPTIONAL: REGISTER TENSOR WITH T2 IN ATLAS SPACE

We can do it using FA as a surrogate to register with t2 image using ANTS. TO BE DONE.

SUMMARY:

- 1) `sh ~/scripts/fetalDTI/dtiTemplate.sh FCB007s2`
- 2) `<copy dirs to DICOM>`
- 3) `python3.6 /home/ch191070/scripts/fetalDTI/prepareDWI.py
/common/projects/Shadab/fetal/Brain/FCB007s2/`
- 4) `<select a reference for B0 image and crop>`
- 5) `python3.5 /home/ch191070/scripts/fetalDTI/createB0B1.py
/common/projects/Shadab/fetal/Brain/FCB007s2/volumes/18_vol_0000_crop.nii.gz`
- 6) `sh /home/ch191070/scripts/fetalDTI/createAtlasT2andMaskFile.sh
mask_cr3Drecon_CASEIDScanNum.nii register_CASEIDScanNum .nii.gz
tfm_nxbmcr3Drecon_CASEIDScanNum_FLIRTto_SOME_REF_FILE.txt CASEIDScanNum
fetus_XX.nii`

7) <Copy T2 data>

8) `python3.6 /home/ch191070/scripts/fetalDTI/registerB0B1toT2.py`
`/common/projects/Shadab/fetal/Brain/CASEIDScanNum/b0b1/dwi b0 CASEIDScanNum.nii.gz`

9) `python3.6 /home/ch191070/scripts/fetalDTI/doSVRandTensorCompute.py`
`/common/projects/Shadab/fetal/Brain/CASEIDScanNum/b0b1/dwi b0 CASEIDScanNum`
`.nii.gz auster b0mi`

10) <refine tensor mask>

11) <Multiply mask>